

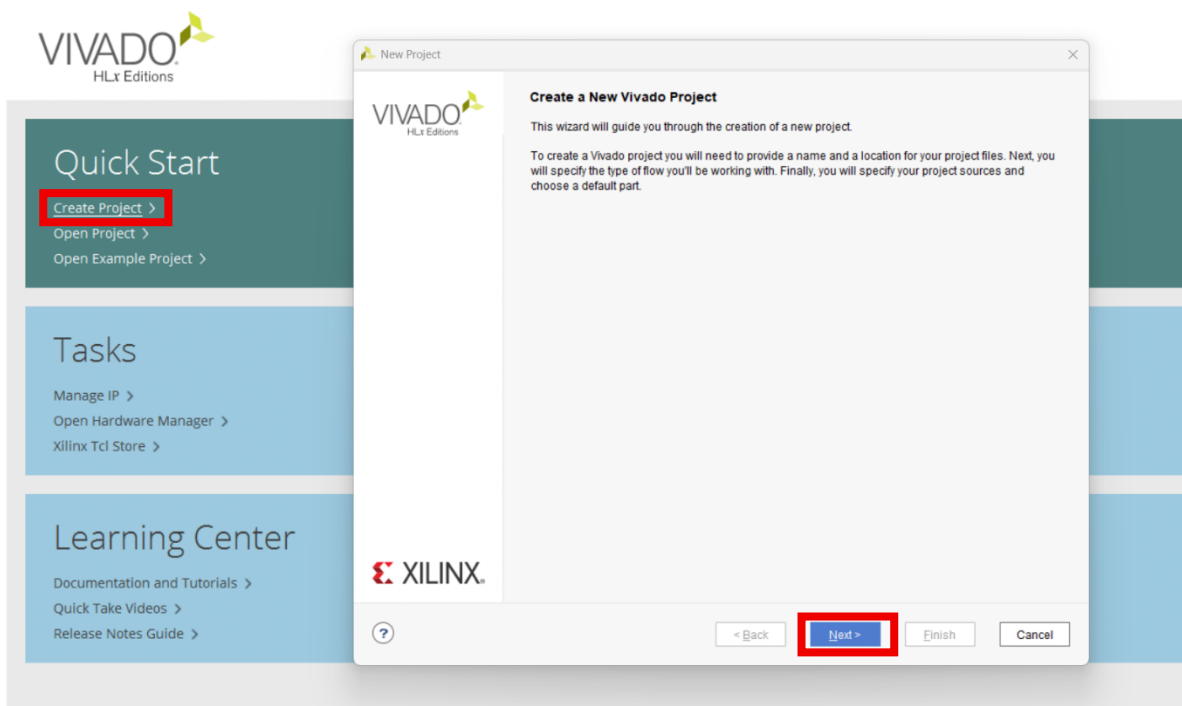
# Lab 1

## Objective of this lab

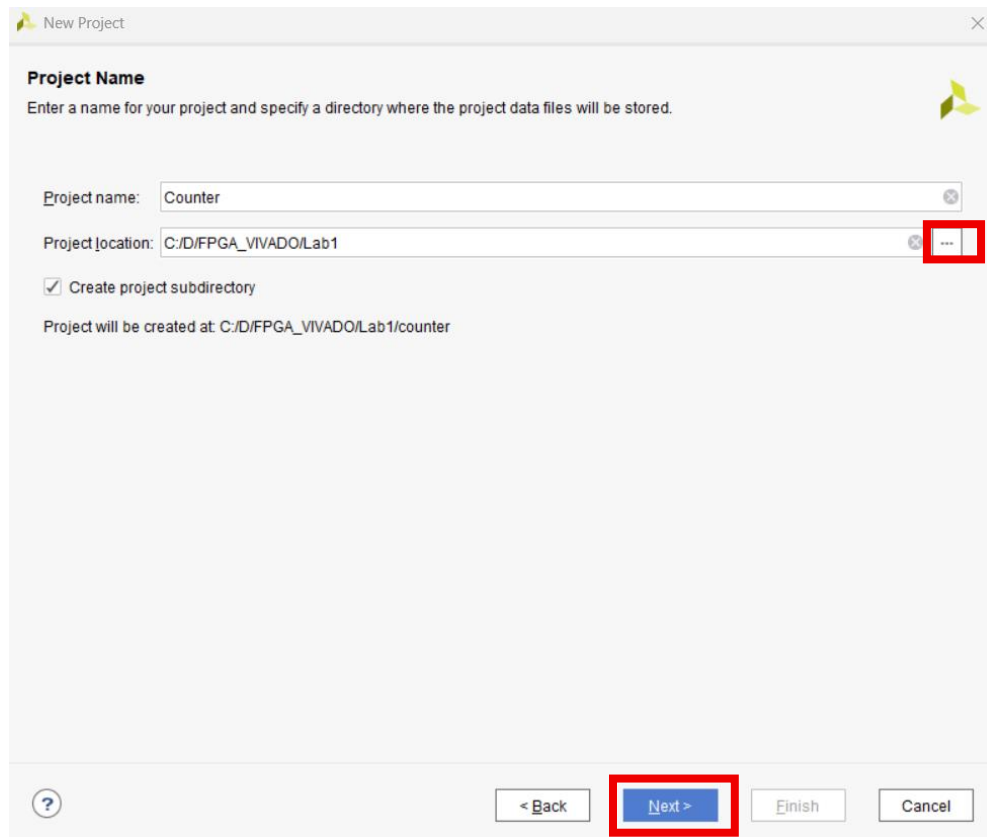
- We will go through the normal Vivado flow and apply it on our Up Down Counter. But to make it visible on FPGA, we will make the counter be 28 bit and then our output interface will represent its 4 MSB.

## Steps: -

1. You will find Counter RTL in folder rtl and I/O pin mapping in folder xdc
2. Launch Vivado and create a new project



### 3. Specify the project's name and its location (click on the 3 points to specify the location)



The 'New Project' dialog box is shown. The 'Project Name' field contains 'Counter'. The 'Project location' field contains 'C:/D/FPGA\_VIVADO/Lab1'. A red box highlights the three dots (browse button) to the right of the 'Project location' field. Below the fields, there is a checked checkbox for 'Create project subdirectory' and a line indicating the project will be created at 'C:/D/FPGA\_VIVADO/Lab1/counter'. At the bottom, there are buttons for '< Back', 'Next >', 'Finish', and 'Cancel'. A red box highlights the 'Next >' button.

**Project Name**  
Enter a name for your project and specify a directory where the project data files will be stored.

Project name: Counter

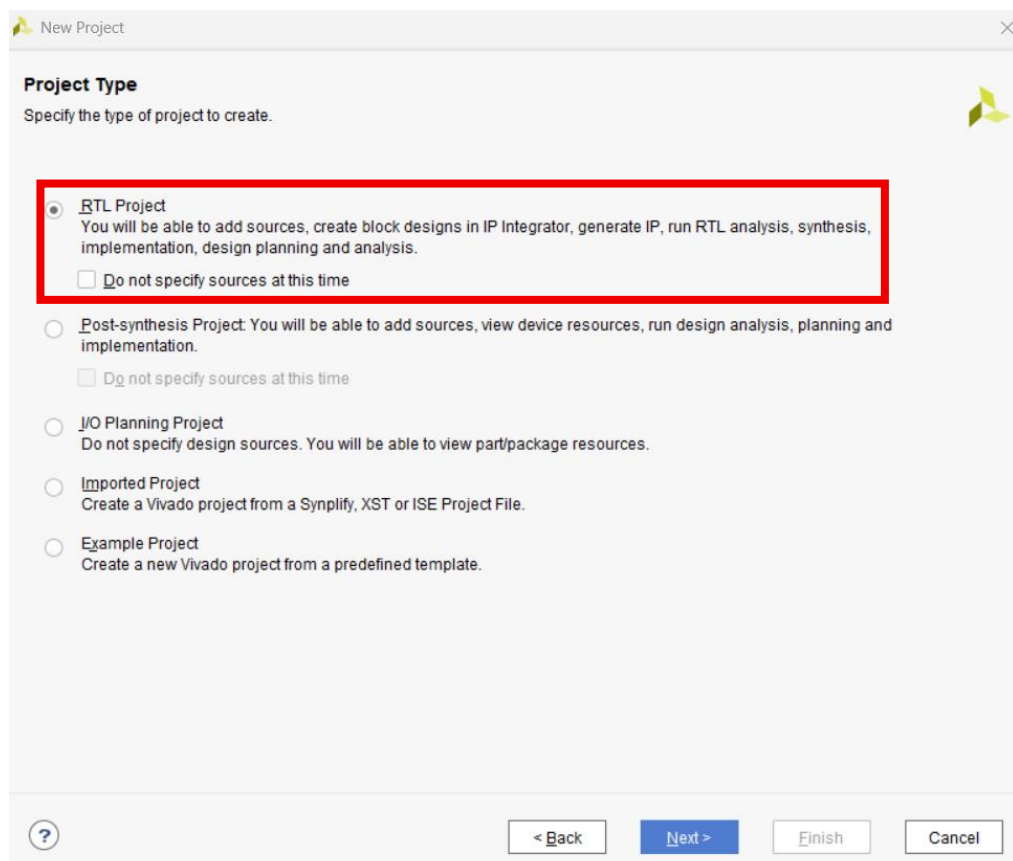
Project location: C:/D/FPGA\_VIVADO/Lab1

☒ Create project subdirectory

Project will be created at: C:/D/FPGA\_VIVADO/Lab1/counter

< Back Next > Finish Cancel

### 4. Select **RTL project** as the project type



The 'New Project' dialog box is shown, now on the 'Project Type' tab. The instruction says 'Specify the type of project to create.' There are five radio button options. The first option, 'RTL Project', is selected and highlighted with a red box. Its description is: 'You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.' Below it is an unchecked checkbox 'Do not specify sources at this time'. The other options are 'Post-synthesis Project', 'I/O Planning Project', 'Imported Project', and 'Example Project', each with a brief description. At the bottom, there are buttons for '< Back', 'Next >', 'Finish', and 'Cancel'.

**Project Type**  
Specify the type of project to create.

☒ **RTL Project**  
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.  
☐ Do not specify sources at this time

☐ Post-synthesis Project: You will be able to add sources, view device resources, run design analysis, planning and implementation.  
☐ Do not specify sources at this time

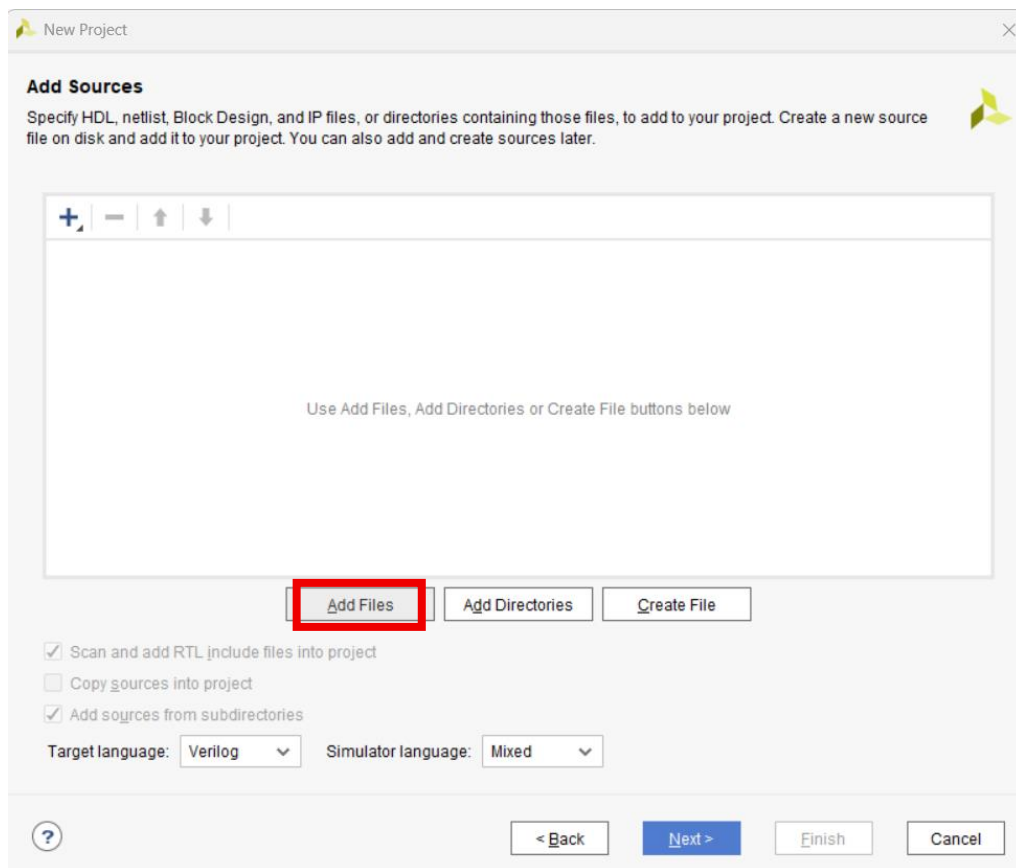
☐ I/O Planning Project  
Do not specify design sources. You will be able to view part/package resources.

☐ Imported Project  
Create a Vivado project from a Synplify, XST or ISE Project File.

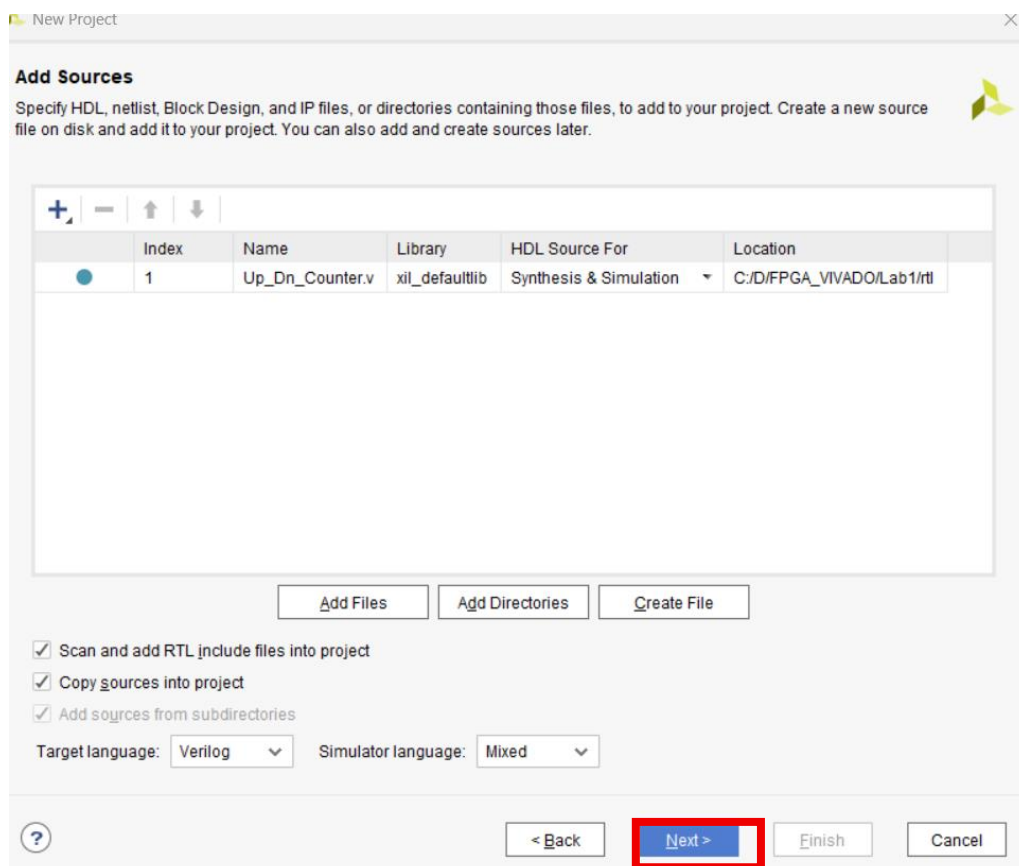
☐ Example Project  
Create a new Vivado project from a predefined template.

< Back Next > Finish Cancel

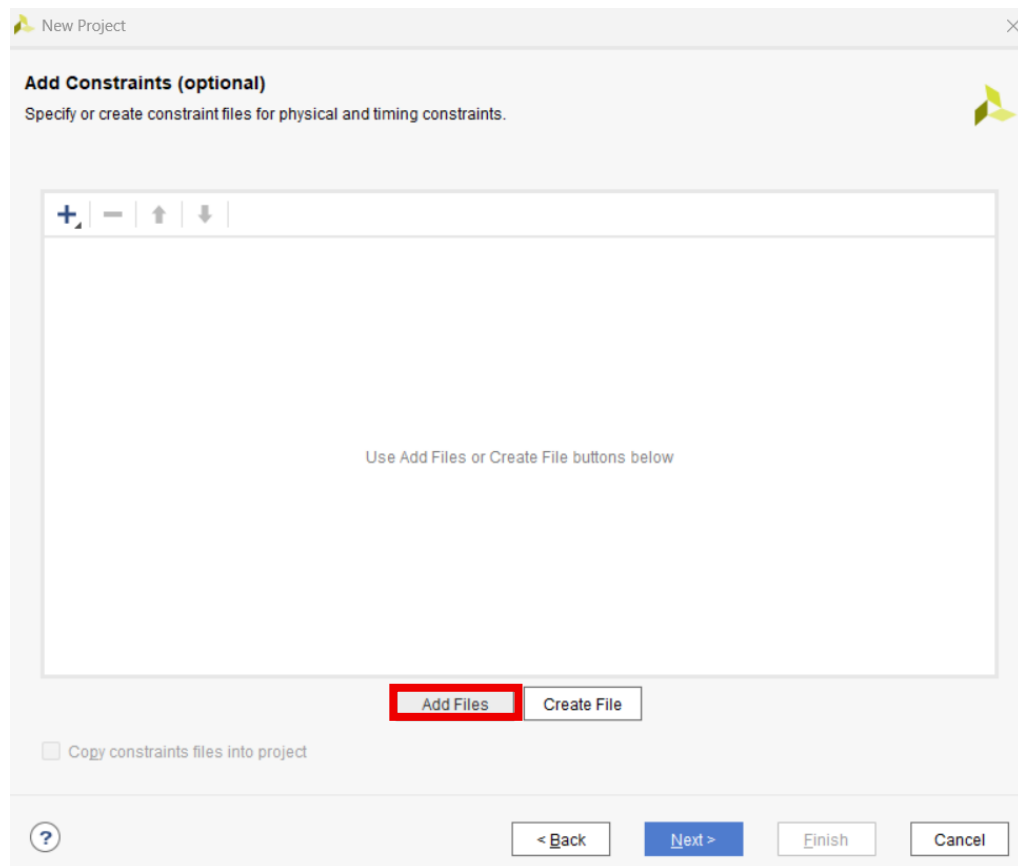
## 5. Add your RTL source files (click on Add Files)



- After adding your RTL files, it will be shown as below:



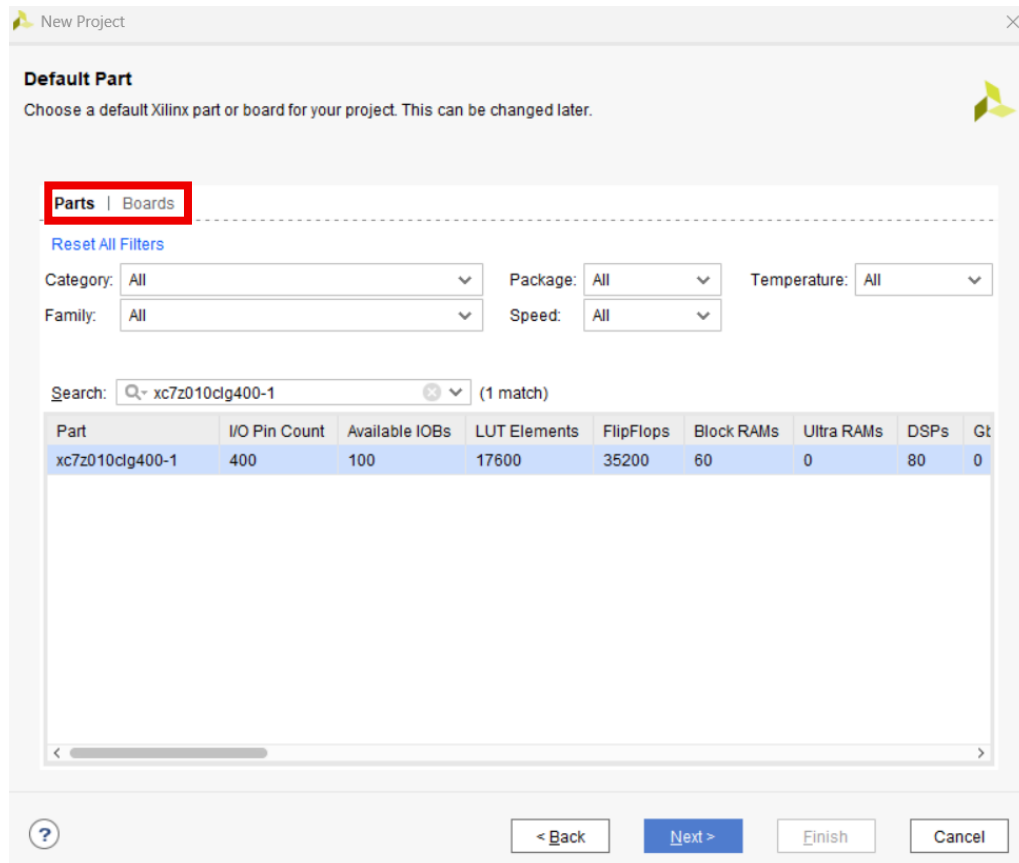
## 6. Add your **constraints (XDC)** file (click on Add Files)



- Your constraints file will have the input/output pin mapping and clock constraints. So, it will be like the following Pic:

```
1  ## Clock Signal (125 MHz)
2  set_property -dict { PACKAGE_PIN K17   IOSTANDARD LVCMOS33 } [get_ports { clk }];
3  create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports { clk }];
4
5  ## Data Input IN[3:0] (Switches)
6  set_property -dict { PACKAGE_PIN G15   IOSTANDARD LVCMOS33 } [get_ports { Up }]; # sw[0]
7  set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { Down }]; # sw[1]
8  set_property -dict { PACKAGE_PIN W13   IOSTANDARD LVCMOS33 } [get_ports { rst_n }]; # sw[2]
9
10
11 ## Visible Counter MSBs (4 On-Board LEDs)
12 set_property -dict { PACKAGE_PIN M14   IOSTANDARD LVCMOS33 } [get_ports { LED_Counter[0] }]; # led[0]
13 set_property -dict { PACKAGE_PIN M15   IOSTANDARD LVCMOS33 } [get_ports { LED_Counter[1] }]; # led[1]
14 set_property -dict { PACKAGE_PIN G14   IOSTANDARD LVCMOS33 } [get_ports { LED_Counter[2] }]; # led[2]
15 set_property -dict { PACKAGE_PIN D18   IOSTANDARD LVCMOS33 } [get_ports { LED_Counter[3] }]; # led[3]
16
17 ## Limit Flags (RGB LED 6)
18 set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports { High }]; # led6_r (Red = Full)
19 set_property -dict { PACKAGE_PIN F17   IOSTANDARD LVCMOS33 } [get_ports { Low }]; # led6_g (Green = Zero)
```

## 7. Select the target FPGA board (xc7z010clg400-1)

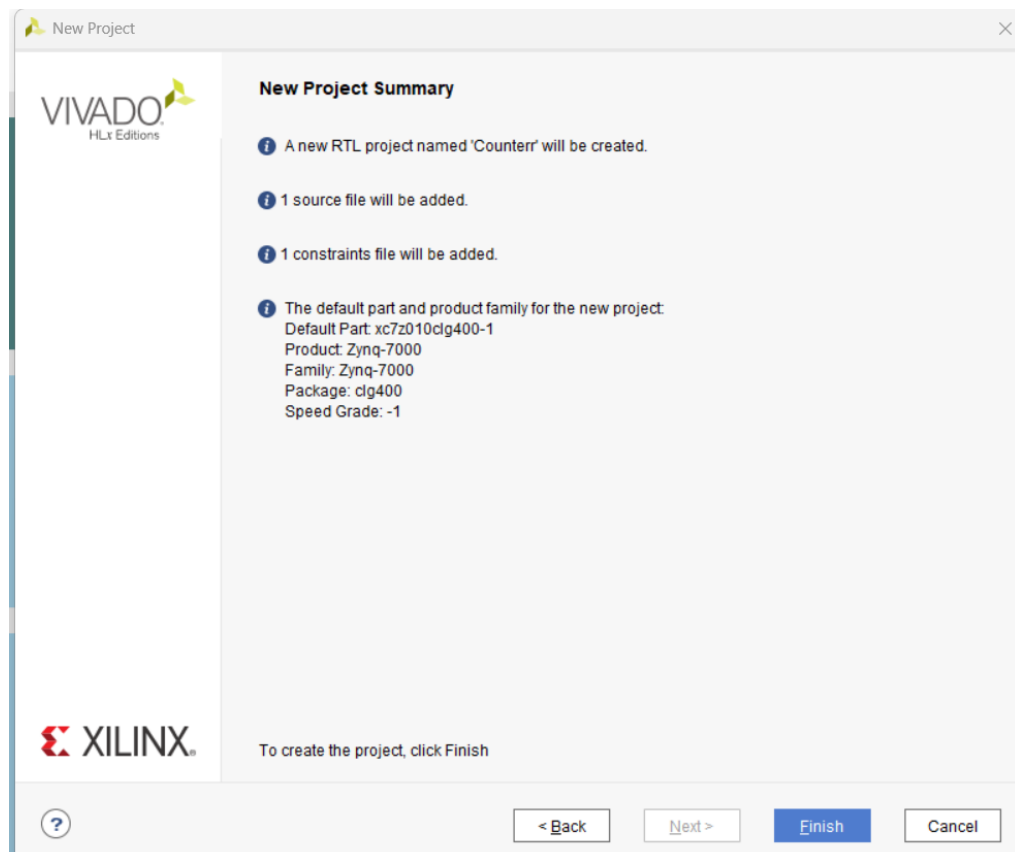


The screenshot shows the 'New Project' dialog box in Vivado, specifically the 'Default Part' selection screen. The 'Parts' tab is selected and highlighted with a red box. Below the tabs, there are filter options: 'Reset All Filters', 'Category: All', 'Family: All', 'Package: All', 'Speed: All', and 'Temperature: All'. A search bar contains the text 'xc7z010clg400-1' and shows '(1 match)'. Below the search bar is a table with the following data:

| Part            | I/O Pin Count | Available IOBs | LUT Elements | FlipFlops | Block RAMs | Ultra RAMs | DSPs | Gt |
|-----------------|---------------|----------------|--------------|-----------|------------|------------|------|----|
| xc7z010clg400-1 | 400           | 100            | 17600        | 35200     | 60         | 0          | 80   | 0  |

At the bottom of the dialog, there are navigation buttons: '< Back', 'Next >', 'Finish', and 'Cancel'.

## 8. Review of the selected project settings (click finish after your confirmation)

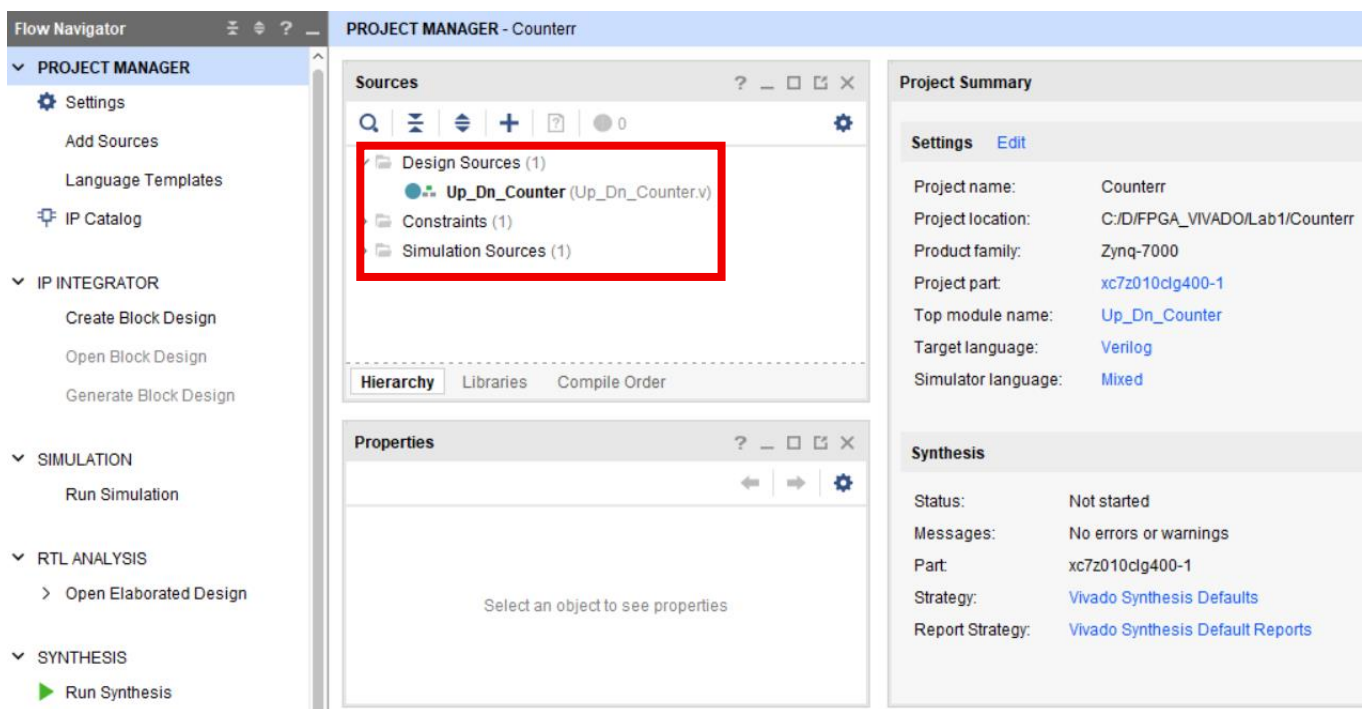


The screenshot shows the 'New Project Summary' screen in Vivado. The left sidebar displays the 'VIVADO' logo and 'HLt Editions'. The main area contains the following summary information:

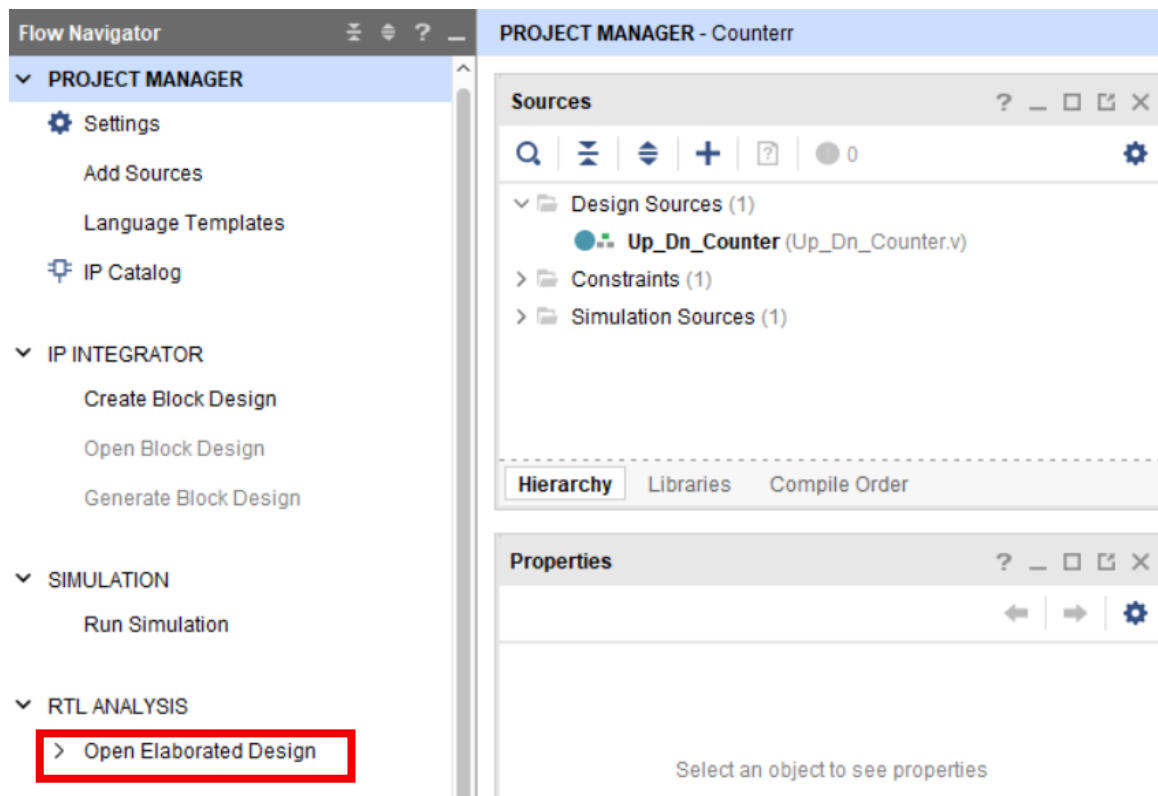
- A new RTL project named 'Counter' will be created.
- 1 source file will be added.
- 1 constraints file will be added.
- The default part and product family for the new project:
  - Default Part: xc7z010clg400-1
  - Product: Zynq-7000
  - Family: Zynq-7000
  - Package: clg400
  - Speed Grade: -1

At the bottom, the XILINX logo is displayed, and the text 'To create the project, click Finish' is shown. Navigation buttons at the bottom include '< Back', 'Next >', 'Finish', and 'Cancel'.

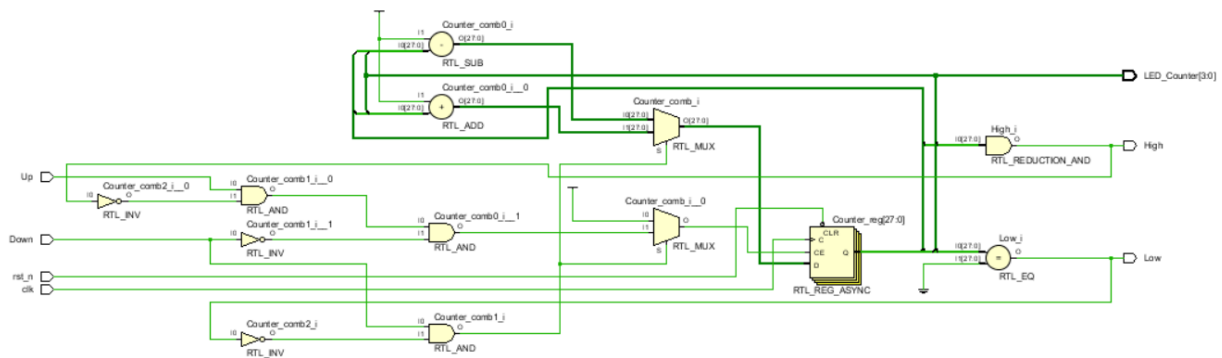
- The next window will be shown as the main interface for Vivado



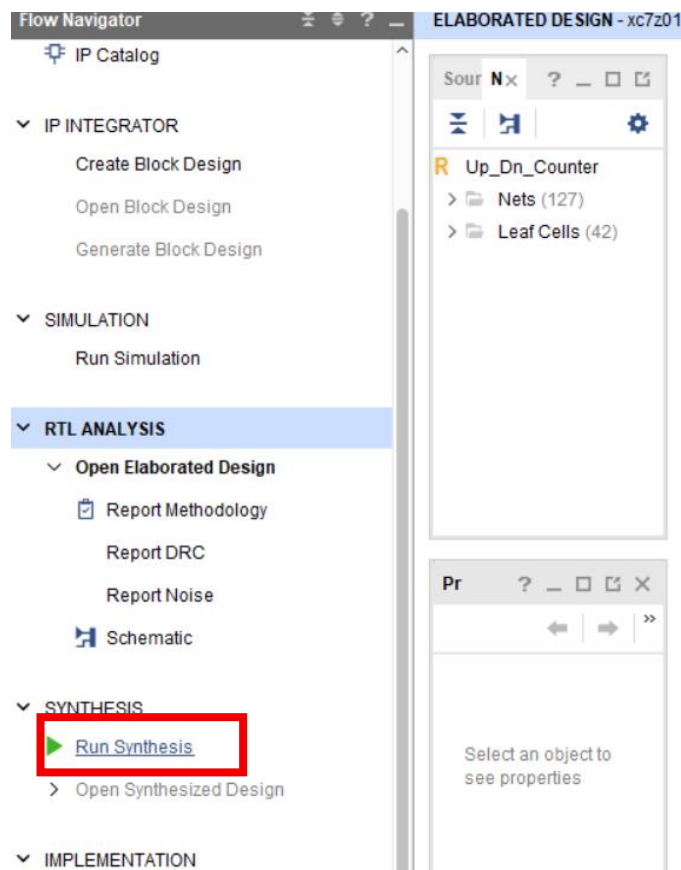
## 9. Run Elaboration



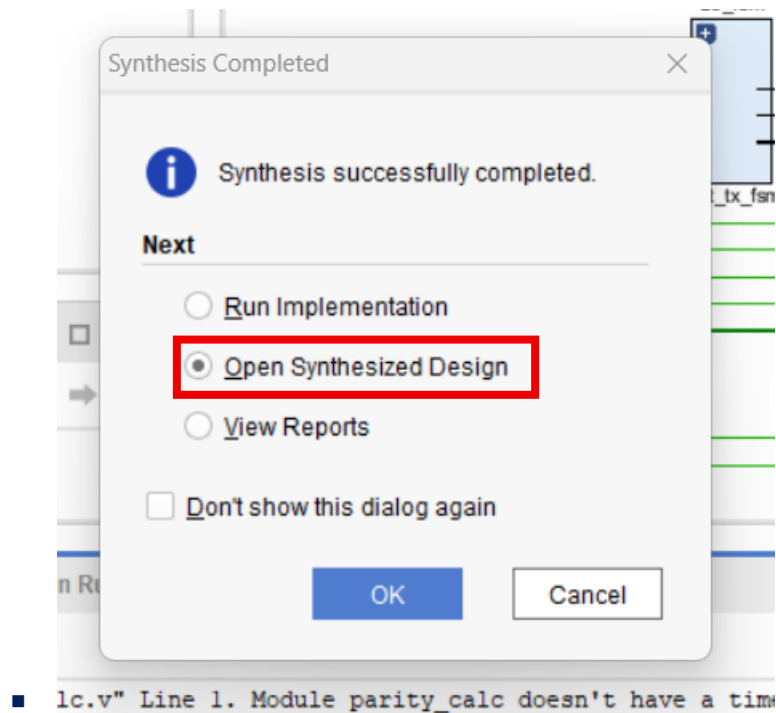
- After you run elaboration, you will see the following high-level schematic for your design



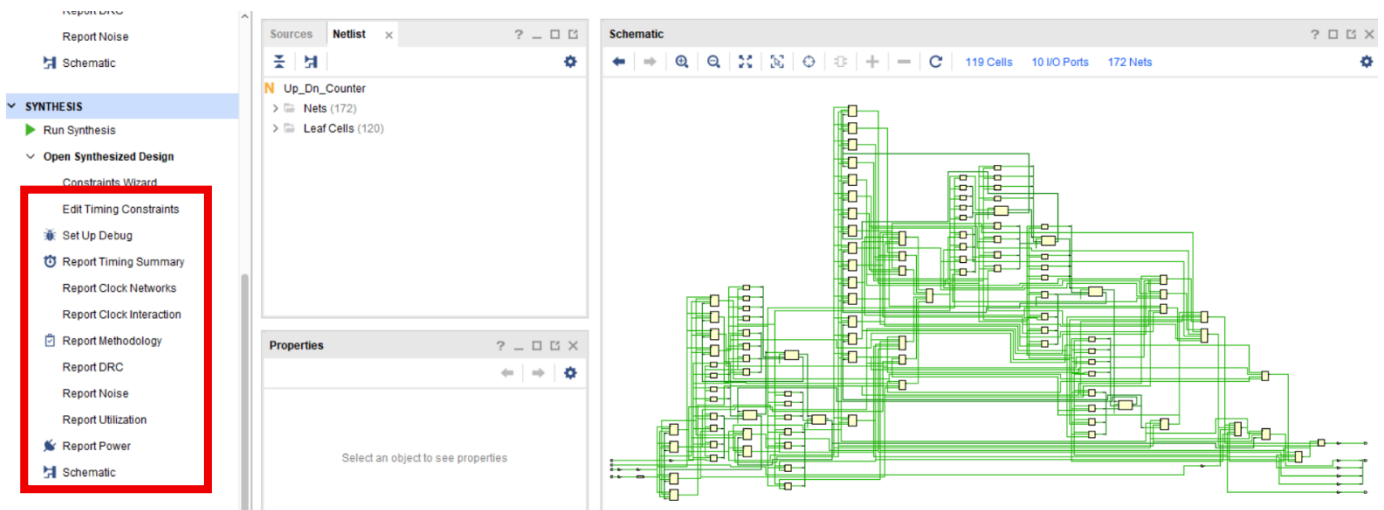
10. Run **Synthesis** to convert the RTL into a synthesized netlist (click on Run Synthesis)



- Open the synthesized design



- The following more detailed schematic will be shown



- There are many reports and options available in synthesis you can select

- **Report timing summary** to check setup and hold slacks

The top screenshot shows the 'Report Timing Summary' dialog box in the IDE. The 'Results name' is 'timing\_1'. Under 'Options', the 'Report' tab is selected. The 'Path delay type' is 'min\_max'. The 'Report unconstrained paths' checkbox is checked. The 'Path Limits' section shows 'Maximum number of paths per clock or path group' set to 10 and 'Maximum number of worst paths per endpoint' set to 1. The 'Path Display' section shows 'Display paths with slack less than' set to 'Use default (1e+30)' and 'Significant digits' set to 3. The 'Command' field contains: `report_timing_summary -delay_type min_max -report_unconstrained -check_timing_verbos -max_paths 10 -input_pins -routable_nets -name timing_1`. The 'Open in a new tab' checkbox is checked.

The bottom screenshot shows the 'Design Timing Summary' report. It includes a table with the following data:

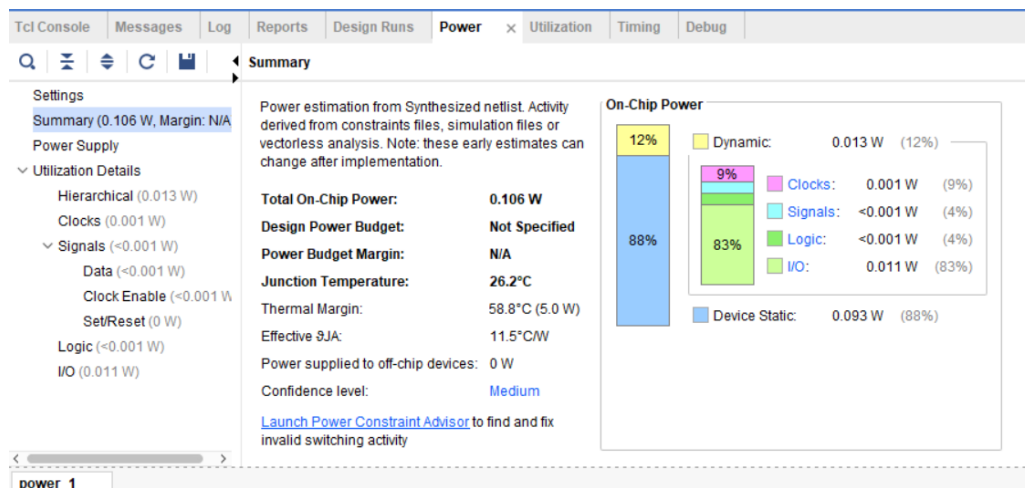
| Setup                                | Hold                             | Pulse Width                                       |
|--------------------------------------|----------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): 2.570 ns | Worst Hold Slack (WHS): 0.201 ns | Worst Pulse Width Slack (WPWS): 3.500 ns          |
| Total Negative Slack (TNS): 0.000 ns | Total Hold Slack (THS): 0.000 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 0       | Number of Failing Endpoints: 0   | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 56        | Total Number of Endpoints: 56    | Total Number of Endpoints: 29                     |

Below the table, it states: 'All user specified timing constraints are met.'

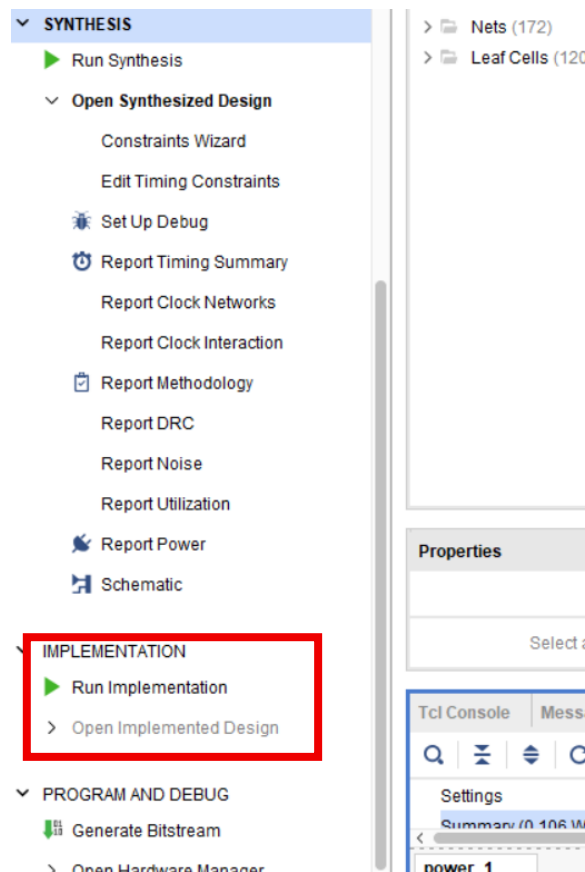
- **Report utilization** shows device resource usage (LUTs, FFs, BRAM, DSP, I/O, clocking) as Used vs. Available

| Hierarchy     |                    |                         |                  |               |  |
|---------------|--------------------|-------------------------|------------------|---------------|--|
| Name          | Slice LUTs (17600) | Slice Registers (35200) | Bonded IOB (100) | BUFGCTRL (32) |  |
| Up_Dn_Counter | 45                 | 28                      | 10               | 1             |  |

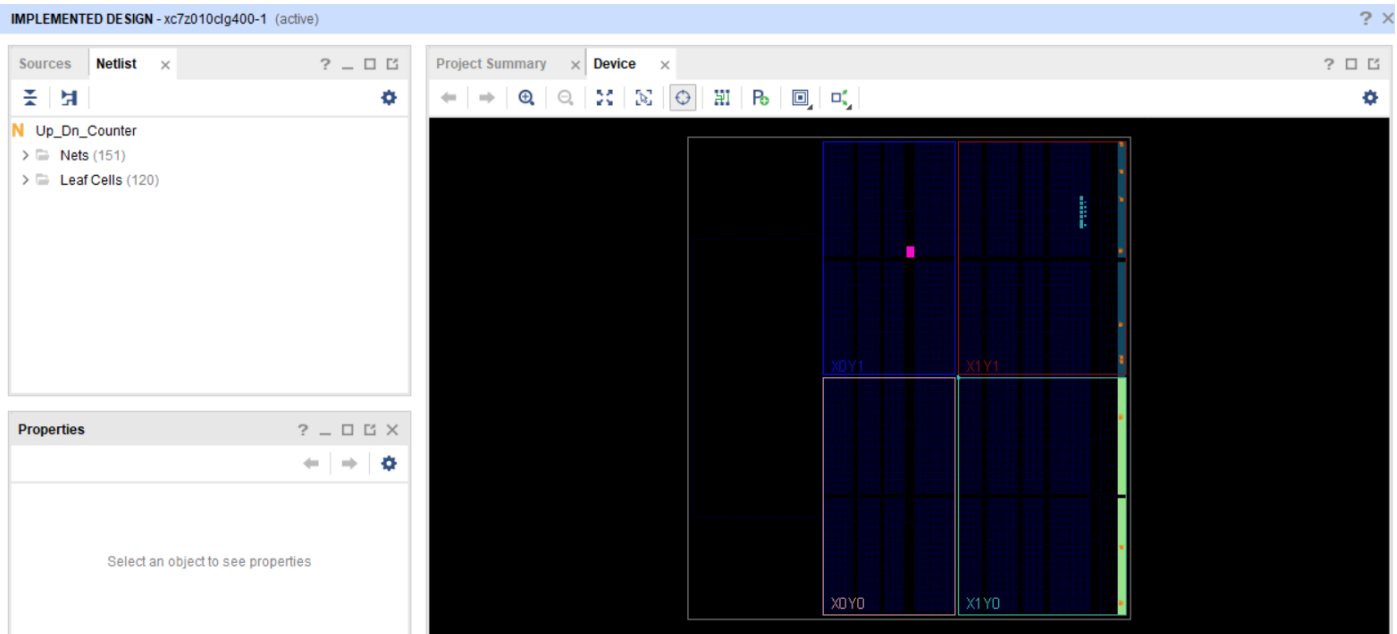
- **Report power** estimates total on-chip power (dynamic + static)



11.Run **Implementation** to perform placement and routing (click in run implementation)

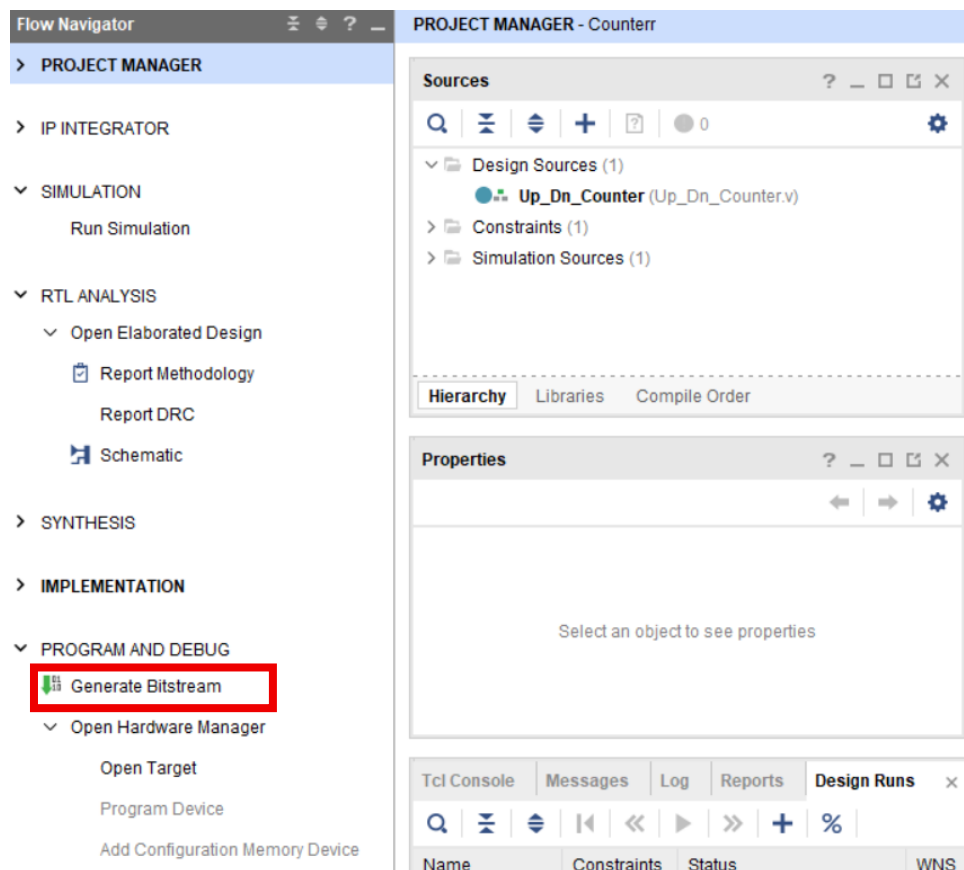


- The following window will be shown:

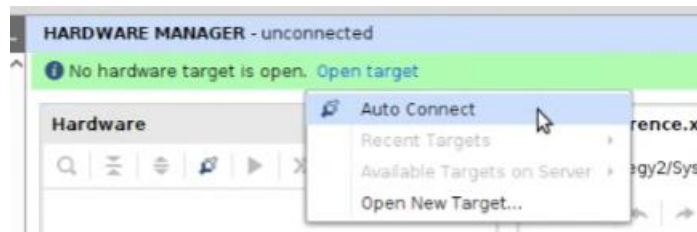


- You can check the same reports mentioned in synthesis but after implementation step.

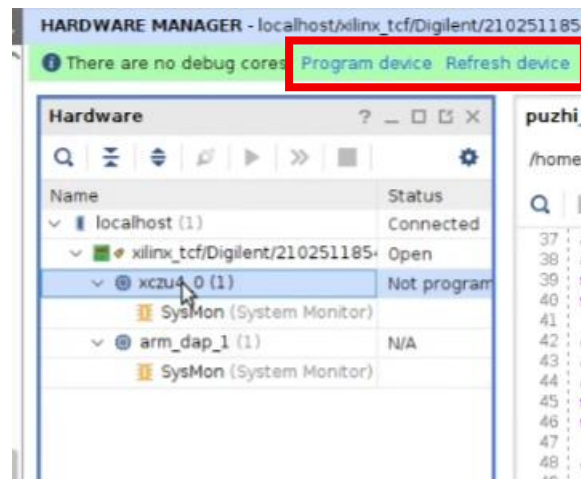
## 12. Generate the **bitstream** and use it to program the FPGA development board



- After **Generate Bitstream** in Vivado, the flow is:
  - Open Hardware Manager
  - Open Target then Auto Connect



- Vivado should detect your FPGA board through JTAG.
- Program the FPGA
- Click: **Program Device**.



- Select your generated .bit file. Vivado normally finds it automatically.
- Then click **Program**. Your FPGA is now configured with your design

